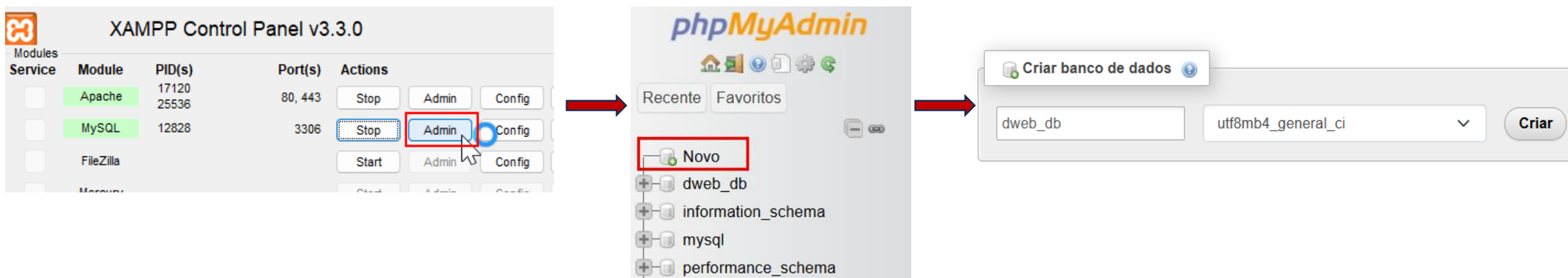


Conexão com banco de dados em PHP 2025/2

FACOM32603 - Desenvolvimento Web I

Prof. Ian R. Cunha
ianresende@ufu.br

Criando o Banco de Dados MySQL



Criando o Banco de Dados MySQL

Nome da Tabela: Número de colunas:

```
1 CREATE TABLE usuarios (  
2   id INT AUTO_INCREMENT PRIMARY KEY,  
3   nome VARCHAR(100) NOT NULL,  
4   email VARCHAR(100) NOT NULL UNIQUE,  
5   senha_hash VARCHAR(255) NOT NULL  
6 );
```

Nome da Tabela: Adicionar: coluna(s)

Nome	Tipo	Tamanho/Valores	Padrão	Colação	Atributos	Nulo	Índice
	INT		Nenhum			☐	
	INT		Nenhum			☐	
	INT		Nenhum			☐	
	INT		Nenhum			☐	

Comentários de tabela: Colação: Mecanismo de armazenamento:

Definição da PARTIÇÃO:

Partição por: (Expressão ou lista de coluna)

Partições:

#	Nome	Tipo	Colação	Atributos	Nulo	Padrão	Comentários	Extra
1	id	int(11)			Não	Nenhum		AUTO_INCREMENT
2	nome	varchar(100)	utf8mb4_general_ci		Não	Nenhum		
3	email	varchar(100)	utf8mb4_general_ci		Não	Nenhum		
4	senha_hash	varchar(255)	utf8mb4_general_ci		Não	Nenhum		

☐ Marcar todos Com marcados:

Acesso ao Banco de Dados MySQL

- Existem duas formas mais comuns para acessar o banco de dados:

MySQLi

- Exclusiva para bancos de dados MySQL/MariaDB.
- Suporta tanto o estilo procedural quanto orientado a objetos.
- Pode ser um pouco mais rápida e ter acesso a recursos muito específicos do MySQL.

PDO (PHP Data Objects)

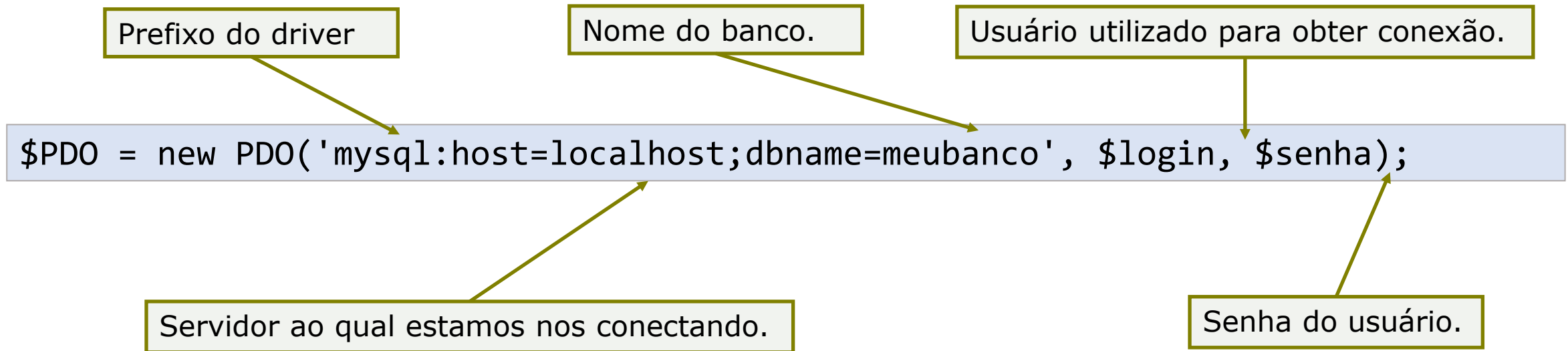
- Abstração de bancos de dados. Funciona com mais de 12 bancos diferentes.
- Apenas orientada a objetos.
- Código mais portátil. Grande parte dos frameworks modernos usa PDO.

PHP Data Objects (PDO)

- Camada de abstração de acesso a dados
 - Mesmas funções independentemente do BD utilizado
- Mais segurança
- API orientada a objetos
- Um pouco mais lenta que a `mysqli`

Conexão com o Banco de Dados com PDO

- É necessário definir os parâmetros de acesso ao BD. Para isso, crie a string de conexão



A conexão com PDO

- O arquivo **conexao.php**.
- A melhor prática é isolar a lógica de conexão em um único arquivo para ser reutilizado.

```
D > conexao.php
1  <?php
2  $host = 'localhost';
3  $user = 'root';
4  $pass = '';
5  $dbname = 'test';
6  $charset = 'utf8mb4';
7
8  $dsn = "mysql:host=$host;dbname=$dbname;charset=$charset";
9  $options = [
10     PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,
11     PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
12     PDO::ATTR_EMULATE_PREPARES   => false,
13 ];
14
15 try {
16     $pdo = new PDO($dsn, $user, $pass, $options);
17 } catch (\PDOException $e) {
18     throw new \PDOException($e->getMessage(), (int)$e->getCode());
19 }
20 ?>
```

Ataques de SQL Injection

- Acontece quando um invasor insere consultas/comandos SQL em um input do usuário.

```
$email = $_POST['email']; // Veio do usuário  
$sql = "SELECT * FROM usuarios WHERE email = '$email'";
```

-> ' OR '1' = '1' --

SELECT * FROM usuarios WHERE email = ' ' OR '1'='1' --'

-> 1; DROP TABLE usuarios

SELECT * FROM usuarios WHERE email = 1; DROP TABLE usuarios

Prepared Statements (Consultas Preparadas)

- A instrução SQL e os dados do usuário viajam em pacotes separados para o banco.
- O banco sabe que os dados são apenas dados, e não comandos.

O Processo de 3 Passos:

- **PREPARE** (Preparar): Prepara o “modelo” da query para o banco, usando “?”.
 - **EXECUTE/BIND**: Envia os "ingredientes" (dados do usuário) de forma segura.
- **FETCH** (Buscar): Pega o resultado.

```
$sql = "SELECT * FROM usuarios WHERE email = ?";  
$stmt = $pdo->prepare($sql);  
$stmt->execute([$email_do_usuario]);  
$usuario = $stmt->fetch();
```

Inserindo dados no BD com PDO

- Exemplo:

```
$sql = "INSERT INTO contato (nome, email) VALUES (?, ?)";
```

```
$stmt = $PDO->prepare($sql);  
$stmt->bindParam(1, $nome);  
$stmt->bindParam(2, $email);
```

Prepara o SQL, em formato de template, para ser executado. Pode conter zero ou mais marcadores de parâmetro (?).

```
if ($stmt->execute()) {  
    if ($stmt->rowCount() > 0) {  
        echo "Nova tupla inserida com sucesso!";  
    } else {  
        echo "Erro ao tentar efetivar cadastro";  
    }  
}
```

Recuperando dados do BD com PDO

- Exemplo:

```
$stmt = $PDO->prepare("SELECT * FROM contatos");

if ($stmt->execute()) {
    while ($rs = $stmt->fetch(PDO::FETCH_OBJ)) {
        echo $rs->nome;
        echo $rs->email;
    }
}
```

Exercícios

1. Crie um novo banco de dados com uma tabela “usuarios” com a seguinte estrutura:
 - cpf varchar(11) not null primary key
 - nome varchar(100) not null
 - email varchar(100) not null UNIQUE
 - senha varchar(255) not null
2. Altere o último exercício da aula anterior (login), criando uma página com formulário para cadastro de usuário. Salve os dados no banco de dados.
 - A senha deve ser salva de forma criptografada. Use as funções corretas do PHP.
3. Ainda no mesmo exercício, altere a página de login para que os dados sejam recuperados do banco de dados e validados corretamente.

Acesso ao Banco de Dados MySQLi

- Existe uma biblioteca chamada mysqli que oferece funções para trabalhar com o MySQL
- Antes de efetuar as consultas no BD, é necessário abrir uma conexão com o SGBD
 - Utilize a função `mysqli_connect()`
- API orientada a objetos e também procedural

Acesso ao Banco de Dados MySQLi

- É necessário definir os parâmetros de acesso ao BD. Para isso, crie a string de conexão
- Exemplo – padrão procedural:

```
$conexao = mysqli_connect  
("localhost", "root", "admin", "meubanco", 3306);
```

Servidor ao qual estamos nos conectando.

Usuário utilizado para obter conexão.

Senha do usuário.

Nome do banco.

Porta onde se conectará no servidor. 3306 é porta padrão do MySQL.

Acesso ao Banco de Dados MySQLi

- É necessário definir os parâmetros de acesso ao BD. Para isso, crie a string de conexão
- Exemplo – padrão OO:

```
$conexao = new mysqli("localhost", "root", "admin", "meubanco", 3306);
```

Servidor ao qual estamos nos conectando.

Senha do usuário.

Porta onde se conectará no servidor. 3306 é porta padrão do MySQL.

Nome do banco.

Usuário utilizado para obter conexão.

Acesso ao Banco de Dados MySQLi

- A função `mysqli_connect` retorna um identificador ou `false`

Procedural

```
if($conexao){  
    echo "Conectei no BD!";  
} else {  
    echo "Ops, deu erro! " . mysqli_connect_error();  
}
```

OO

```
if($conexao->connect_errno){  
    echo "Conectei no BD!";  
} else {  
    echo "Ops, deu erro! " . $mysqli->connect_error;  
}
```


Inserção de dados no BD - MySQLi

- A função `mysqli_query()` é utilizada para executar consultas DML no BD.

Linguagem de Manipulação de Dados: INSERT, DELETE e UPDATE

- No caso de inserção, retorna *true*, em caso de sucesso, ou *false*, caso contrário

```
$sql = "INSERT INTO ... ";
```

```
if (mysqli_query($conexao, $sql)) {  
    echo "Nova tupla inserida com sucesso!";  
} else {  
    echo "Erro: " . $sql . "<br>" . mysqli_error($conexao);  
}
```

Padrão OO:

```
if ($result = $mysqli->query("INSERT INTO ...")) { ... }
```

Recuperando dados do BD - MySQLi

- Utilize a função `mysqli_query()`
 - Essa função retorna um objeto `mysqli_result` (conjunto de linhas) em caso de sucesso de consultas que buscam dados
- Exemplos:

```
$rsClientes = mysqli_query($conexao, "SELECT * FROM clientes");
```

```
$rsProdutos = mysqli_query($conexao, "SELECT * FROM produtos");
```

Padrão OO:

```
$rsProdutos = $mysqli->query("SELECT * FROM produtos")) { ... }
```

Recuperando dados do BD - MySQL

Há outras funções que também podem ser utilizadas, como `mysqli_fetch_all()`, `mysqli_fetch_object()`, ...

- Para ler o conteúdo de uma linha do conjunto de dados retornado, utilizamos a função `mysqli_fetch_assoc()`, passando o objeto `mysqli_result` como parâmetro
- Exemplo:

```
while($cliente = mysqli_fetch_assoc($rsClientes)){  
    $varNome = $cliente["nome"];  
}
```

Pega cada linha do conjunto retornado pela consulta e coloca em um array associativo `$cliente`

Nome da coluna no BD

Padrão OO:
`$row = $result->fetch_array(MYSQLI_ASSOC);`

Recuperando dados do BD - MySQLi

- Para ler o conteúdo de uma linha do conjunto de dados retornado, utilizamos a função `mysqli_fetch_assoc()`, passando o objeto `mysqli_result` como parâmetro
- Exemplo:

Cria um array para guardar os clientes

```
$clientes = array();  
$i = 0;  
while($cliente = mysqli_fetch_assoc($rsClientes)){  
    $clientes[$i] = $cliente;  
    $i++;  
}
```

Pega cada linha do rowset retornado pela consulta

Referências

- <https://www.php.net/docs.php>
- <https://www.w3schools.com/php>
- LOCKHART, J. e STURGEON, P. PHP: The Right Way. Lean Pub, 2016. Tradução disponível em <http://br.phptherightway.com>